
ジオメトリの組み込み

演習パッケージ: P06_Geometry

Geant4 11.3準拠

Geant4講習会資料: HEP/NP/Space 2025

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

演習トピックス

[概要] P06_Geometryプログラムを使い、ジオメトリの記述方法を学ぶ

[目次]

1. P06_Geometry/WaterBox
 - ジオメトリの記述方法の基礎を学ぶ
2. P06_Geometry/WaterContainer
 - 容器に入った水の記述
3. P06_Geometry/BGO_One
 - 円筒型BGO検出器ジオメトリ
4. P06_Geometry/BGO_Two
 - 円筒型BGO検出器ジオメトリ: Logical Volumeの複数回使用
5. P06_Geometry/Pixel_One
 - Pixel検出器ジオメトリ: Replicaの使用

自習課題: 以下の課題は演習で扱わない。演習終了後、各自で実行してみる

- P06_Geometry/WaterExtruded: ボリュームが部分重複したらどうなるか
- P06_Geometry/Pixel_Four: 4台のPixel検出器の配置

P06_Geometry/WaterBox

ジオメトリの記述方法の基礎を学ぶ

演習トピックス

[概要] [P06_Geometry](#)プログラムを使い、ジオメトリの記述方法を学ぶ

[目次]



1. [P06_Geometry/WaterBox](#)
 - ジオメトリの記述方法の基礎を学ぶ
2. [P06_Geometry/WaterContainer](#)
 - 容器に入った水の記述
3. [P06_Geometry/BGO_One](#)
 - 円筒型BGO検出器ジオメトリ
4. [P06_Geometry/BGO_Two](#)
 - 円筒型BGO検出器ジオメトリ: Logical Volumeの複数回使用
5. [P06_Geometry/Pixel_One](#)
 - Pixel検出器ジオメトリ: Replicaの使用

前演習プログラムを再現する

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- 前演習(P05_FirstStep/WaterBox)の全コードがP06_Geometry/WaterBoxにコピーされている

課題:2 P06_Geometry/WaterBoxのソースコードをビルドして実行する

1) P06_Geometry/WaterBoxのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P06_Geometry/WaterBox
$ ls
source/  util/  TestBench/
```

← sourceはP05_FirstStep/WaterBox/sourceと完全同一
TestBenchは前演習と同一のものを組み込み済み

2) ソースをビルド

- 前演習ではビルドからTestBench及びMacro作成を逐次コマンド入力で行う手順を説明した
- ビルド手順は全演習を通して共通なので、ビルド自動実行のscriptをutil/Build.shに用意
- util/Build.shを使い以下のようにソースをビルドすると前演習(FirstStep)の結果が再現される

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ)に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

前演習プログラムを再現する — つづき

課題:3 実行されたプログラムを使う

[注] コマンド入力は「tabキー」補完機能を使う

1) Qt画面のコマンド窓に以下のコマンドを入力

```
/control/execute GlobalSetup.mac  
/run/beamOn 1
```

または、Qtのファイルアイコンをクリック

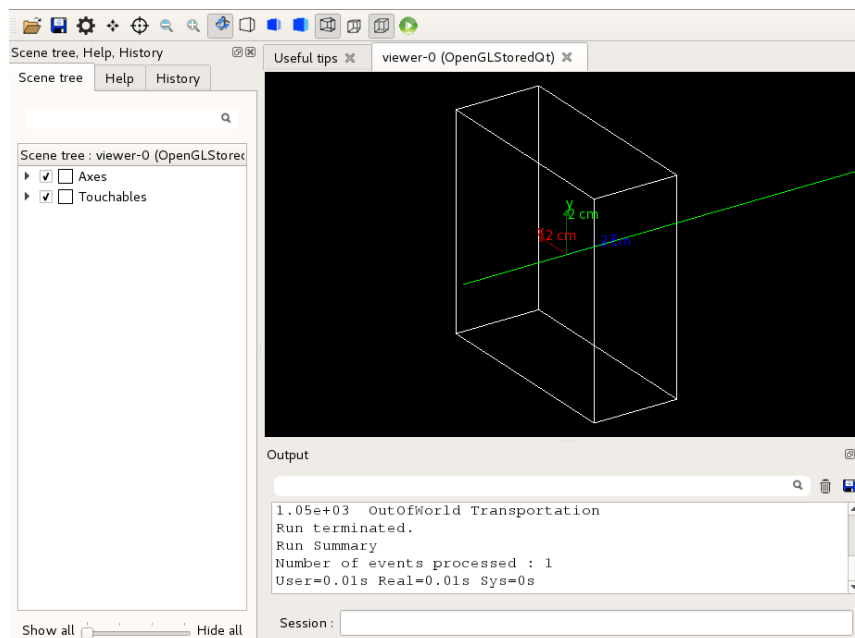
- 右図の様にgenantinoの飛跡が見えることを確認

2) アプリを終了

```
exit
```

[注] 'exit'は先頭に'/ 'が不要

次の課題では、実行したプログラムの
Geometryクラスのコードを読み、実装手
法を学ぶ



P06_Geometry/WaterBox: Geomety.hhの内容

課題:1 WaterBoxジオメトリ定義ヘッダーファイルを理解する

[注] このヘッダーファイルは以後のすべての演習プログラムで共通に使われる

→ また、ユーザが作成するジオメトリ定義では、このヘッダー・ファイルが流用できる

```
$ cd ../source/include  
$ less Geometry.hh
```

ヘッダーファイルのdirに行き、
Geometry.hhを書き出す

```
//+++++  
// Geometry.hh  
//+++++  
#ifndef Geometry_h  
#define Geometry_h 1  
  
#include "G4VUserDetectorConstruction.hh"  
class G4VPhysicalVolume;  
  
//-----  
class Geometry : public G4VUserDetectorConstruction  
//-----  
{  
public:  
    Geometry();  
    ~Geometry() override;  
  
    G4VPhysicalVolume* Construct() override;  
};  
#endif
```

ユーザのジオメトリ定義は必ず
この仮想クラスを継承して行う

Geometryクラスの定義

ユーザのジオメトリ実装コードは
Construct関数の中に記述する

[注] 'override'は基底クラスで定義された
関数内容を上書きすることを宣言
(必須ではないが、宣言の方が安全)

P06_Geometry:WaterBox Geomety.ccの内容

課題:1 WaterBoxのジオメトリ定義のソース ファイルを理解する

```
$ cd ../src  
$ less Geometry.cc
```

ソース ファイルのdirに行き、
Geometry.ccを書き出す

```
//+++++  
// Geometry.cc  
//+++++  
#include "Geometry.hh"  
#include "G4Box.hh"  
#include "G4LogicalVolume.hh"  
#include "G4PVPPlacement.hh"  
#include "G4VPhysicalVolume.hh"  
#include "G4ThreeVector.hh"  
#include "G4RotationMatrix.hh"  
#include "G4Transform3D.hh"  
#include "G4NistManager.hh"  
#include "G4VisAttributes.hh"  
#include "G4SystemOfUnits.hh"
```

Geometry.ccで使うクラス
のヘッダー・ファイルをすべ
てincludeする

```
//-----  
Geometry::Geometry() {}  
//-----  
  
//-----  
Geometry::~Geometry() {}  
//-----
```

```
//-----  
G4VPhysicalVolume* Geometry::Construct()  
//-----  
{  
// Get the pointer to 'Material Manager'  
G4NistManager* materi_Man = G4NistManager::Instance();
```

Construct関数の実装

NISTマテリアル・マネージャ作成
World volumeの作成開始

```
// Define 'World Volume'  
// Define the shape of solid  
G4double leng_X_World = 2.0*m; // X-full-length of world  
G4double leng_Y_World = 2.0*m; // Y-full-length of world  
G4double leng_Z_World = 2.0*m; // Z-full-length of world  
auto solid_World =  
    new G4Box( "Solid_World", leng_X_World/2.0, leng_Y_World/2.0, leng_Z_World/2.0 );
```

Worldのsolid(box)を作成

```
// Define the logical volume  
G4Material* materi_World = materi_Man->FindOrBuildMaterial( "G4_AIR" );  
auto logVol_World = new G4LogicalVolume( solid_World, materi_World, "LogVol_World" );  
logVol_World->SetVisAttributes ( G4VisAttributes::Invisible );
```

Worldのlogical volume(air)を作成

World volumeを配置

```
// Placement of 'World Volume'  
G4int copyNum_World = 0;  
auto physVol_World = new G4PVPPlacement( G4Transform3D(), "PhysVol_World",  
                                           logVol_World, 0, false, copyNum_World );
```

Worldのphysical volumeを作成
(親volumeは存在しないので0を設定)

[注] ソースは次のスライドに続く

P06_Geometry:WaterBox Geomety.ccの内容 — つづき

課題:2 WaterBoxのジオメトリ定義のソース ファイルを理解する — つづき

```
// Define 'Water Box'
// Define the shape of solid
G4double leng_X_WaterBox = 10.0*cm;    // X-full-length of water box
G4double leng_Y_WaterBox = 10.0*cm;    // Y-full-length of water box
G4double leng_Z_WaterBox = 4.0*cm;     // Z-full-length of water box
auto solid_WaterBox = new G4Box{ "Solid_WaterBox", leng_X_WaterBox/2.0,
                                leng_Y_WaterBox/2.0, leng_Z_WaterBox/2.0 };

// Define the logical volume
G4Material* materi_WaterBox = materi_Man->FindOrBuildMaterial( "G4_WATER" );
auto logVol_WaterBox = new G4LogicalVolume{ solid_WaterBox, materi_WaterBox,
                                           "LogVol_WaterBox", 0, 0, 0 };

// Placement of 'Water Box' into 'World Volume'
// Create G4Transform3D to define rotation/translation
G4double pos_X_LogV = 0.0*cm;          // X-location LogV
G4double pos_Y_LogV = 0.0*cm;          // Y-location LogV
G4double pos_Z_LogV = 0.0*cm;          // Z-location LogV
auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV };
auto rotMtrx_LogV = G4RotationMatrix{};
auto trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV };

// Install 'Water Box'
G4int copyNum_LogV = 1000;              // Set ID number of LogV
new G4PVPlacement{ trans3D_LogV, "PhysVol_WaterBox", logVol_WaterBox,
                  physVol_World, false, copyNum_LogV, true };

// Return the physical volume of 'World'
return physVol_World;
}
```

Water Boxの作成開始

Water Boxのsolid(box)を作成

Water Boxのlogical volume(water)を作成

Water Boxの配置開始

Water Box配置の位置情報をG4Transform3Dオブジェクトを使って作成

Water BoxのWorld Volumeへの配置

- Water BoxのvolumeID#(copyNumber)を1000に指定
- G4PVPlacementを使い、Water Boxのlogical volumeをWorldのphysical volumeに設置

worldのphysical volumeのポインターを返す

P06_Geometry/WaterContainer

容器に入った水の記述

演習トピックス

[概要] [P06_Geometry](#)プログラムを使い、ジオメトリの記述方法を学ぶ

[目次]

1. [P06_Geometry/WaterBox](#)
 - ジオメトリの記述方法の基礎を学ぶ
2. [P06_Geometry/WaterContainer](#)
 - 容器に入った水の記述
3. [P06_Geometry/BGO_One](#)
 - 円筒型BGO検出器ジオメトリ
4. [P06_Geometry/BGO_Two](#)
 - 円筒型BGO検出器ジオメトリ： Logical Volumeの複数回使用
5. [P06_Geometry/Pixel_One](#)
 - Pixel検出器ジオメトリ： Replicaの使用

P06_Geometry/WaterContainerプログラムの概要

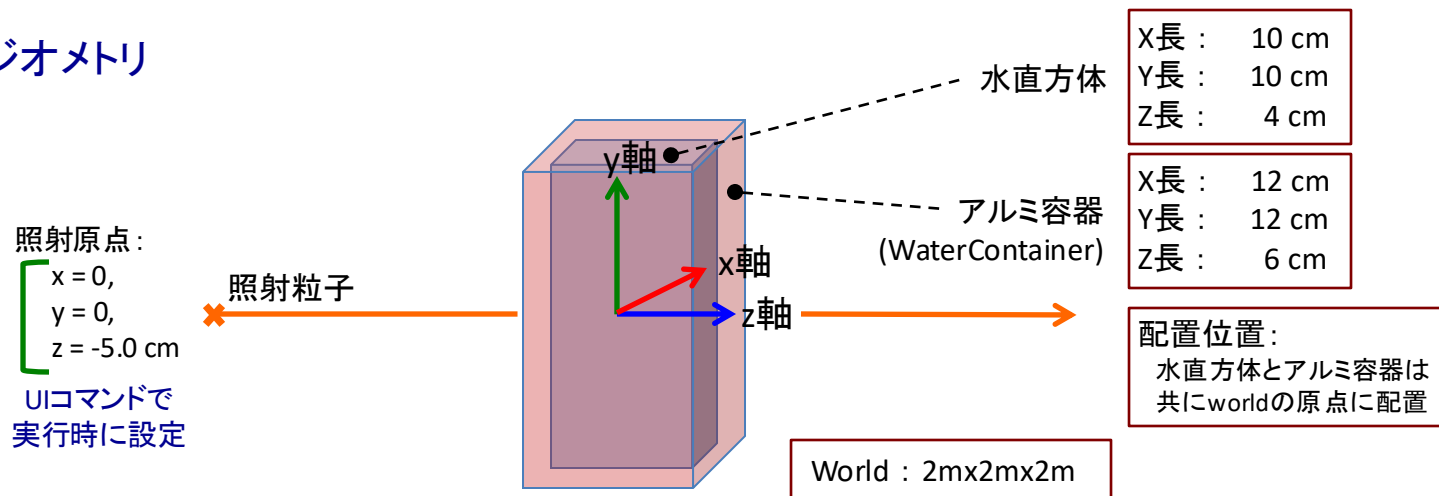
■ 演習プログラムの目的

- 前演習のジオメトリでは水直方体は容器に入っていない非現実的なもの
→ この演習ではアルミ容器 (WaterContainer) 内に水を入れるジオメトリを考える
- 演習を実行することで、logical volumeにlogical volumeを配置する方法を学ぶ

■ プログラムの構成

- mainプログラム: 前演習と同一
- ジオメトリ: 水がアルミ直方体容器の中に入っている(下図参照)
- PhysicsList: 前演習と同一
- PrimaryGenerator: 前演習と同一

■ 組み込むジオメトリ



P06_Geometry/WaterContainerプログラムのビルド

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- 前演習のWaterBoxジオメトリにWaterContainerが追加されたGeometry.ccが組み込まれている
- Geometry.cc以外のコードは前演習と同一

課題:2 P06_Geometry/WaterContainerのソースコードをビルドして実行する

1) P06_Geometry/WaterContainerのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P06_Geometry/WaterContainer
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ)に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P06_Geometry/WaterContainerプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac  
/run/beamOn 1
```

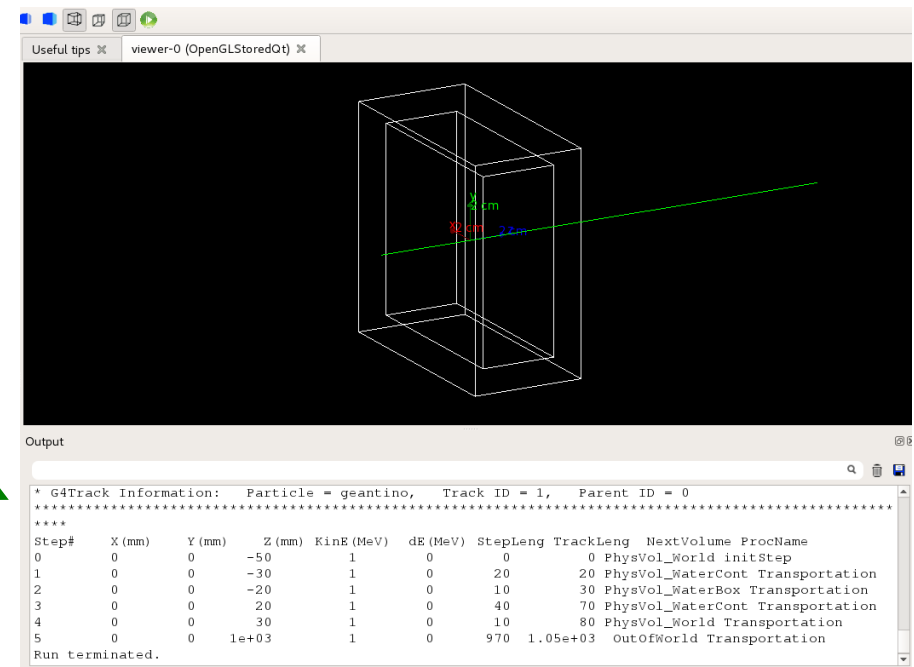
または、Qtのファイルアイコンをクリック

- 右図の様にgeantinoの飛跡が見えることを確認

2) Qtの出力窓に表示されている、geantinoのtracking情報を読んで、trackがアルミコンテナと水boxを正しく通過していることを確認

3) アプリを終了

```
exit
```



次の課題では、実行したプログラムの
Geometryクラスのコードを読み、実装手
法を学ぶ

P06_Geometry/WaterContainer: Geometry.ccの内容

課題:1 前演習(WaterBox)のGeometry.ccとWaterContainerのGeometry.ccをcolordiffコマンドで比較

```
$ cd ../source/src
$ colordiff -y --width=160 ../../WaterBox/source/src/Geometry.cc Geometry.cc
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から新たに追加された箇所を確認する

変更前ファイル		変更後ファイル	
G4VPhysicalVolume* Geometry::Construct() //----- == world volume作成部分のCodeは不変なので省略 == // Placement of 'Water Box' into 'World Volume'		G4VPhysicalVolume* Geometry::Construct() //----- // Define 'Water Container'	
		> // Define the shape of solid	
		> G4double leng_X_WaterCont = 12.0*cm; // X-full-length of water contain	
		> G4double leng_Y_WaterCont = 12.0*cm; // Y-full-length of water contain	
		> G4double leng_Z_WaterCont = 6.0*cm; // Z-full-length of water contain	
		> auto solid_WaterCont = new G4Box("Solid_WaterCont", leng_X_WaterCont/2.0,	
		leng_Y_WaterCont/2.0, leng_Z_WaterCont/2	
		>	
		> // Define logical volume (Container is Aluminun)	
		> G4Material* materi_WaterCont = materi_Man->FindOrBuildMaterial("G4 Al");	
		> auto logVol_WaterCont = new G4LogicalVolume(solid_WaterCont, materi_Water	
		"LogVol_WaterCont", 0, 0, 0 }	
		>	
		> // Placement of 'Water Box' into 'Water Container'	
		> // Create G4Transform3D to define rotation/translation	
		> G4double pos_X_LogV = 0.0*cm; // X-location LogV	
		> G4double pos_Y_LogV = 0.0*cm; // Y-location LogV	
		> G4double pos_Z_LogV = 0.0*cm; // Z-location LogV	
		> auto threeVect_LogV = G4ThreeVector(pos_X_LogV, pos_Y_LogV, pos_Z_LogV);	
		> auto rotMtrx_LogV = G4RotationMatrix{};	
		> auto trans3D_LogV = G4Transform3D(rotMtrx_LogV, threeVect_LogV);	
		>	
		> // Install 'Water Box'	
		> G4int copyNum_LogV = 1000; // Set ID number of LogV	
		> new G4PVPlacement(trans3D_LogV, "PhysVol_WaterBox", logVol_WaterBox, 変更	
		> logVol_WaterCont, false, copyNum_LogV, true);	
		>	
		> // Placement of 'Water Container' into 'World Volume'	
		> // Install 'Water Container' - reuse G4Transform3D for 'Water Box'	
		> copyNum_LogV = 2000; // Set ID number of LogV	
		> new G4PVPlacement(trans3D_LogV, "PhysVol_WaterCont", logVol_WaterCont,	
		physVol_World, false, copyNum_LogV, true);	
		>	
		> // Return the physical volume of 'World'	
		> return physVol_World;	
		>	
		> }	

P06_Geometry/WaterContainerプログラムの留意点

課題:1 WaterContainerプログラムのジオメトリ記述で使われている手法を理解する

- 1) containerの外寸に一致するアルミ・ボリウム(box)を作成し、その中に水・ボリウム(box)を配置することで、二つのボリウムが重なる部分は、水に置き換わる
- 2) これにより、containerの外殻を表現するボリウムを作成しなくても、水を保持するcontainerが表現できる
- 3) アルミ・ボリウムを表現するlogical volumeの中に、水・ボリウムを表現するlogical volumeを配置する
- 4) アルミ・ボリウムlogical volumeをphysical volumeであるWorldボリウムに配置すれば、水・ボリウムも自動的に配置される
- 5) 上の3)と4)のボリウム配置は共にG4PVPlacementを使うが、その関数シグネチャが異なることに留意

→ 以下のコードを参照

関数が持つ引数の内容

WaterContainerプログラムのGeometry.ccの最終部分

```
// Placement of 'Water Box' into 'Water Container'
// Create G4Transform3D to define rotation/translation
G4double pos_X_LogV = 0.0*cm;           // X-location LogV
G4double pos_Y_LogV = 0.0*cm;           // Y-location LogV
G4double pos_Z_LogV = 0.0*cm;           // Z-location LogV
auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV };
auto rotMtrx_LogV = G4RotationMatrix{};
auto trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV };

// Install 'Water Box'
G4int copyNum_LogV = 1000;               // Set ID number of LogV
new G4PVPlacement{ trans3D_LogV, logVol_WaterBox, "PhysVol_WaterBox",
                  logVol_WaterCont, false, copyNum_LogV, true };

// Placement of 'Water Container' into 'World Volume'
// Install 'Water Container' - reuse G4Transform3D for 'Water Box'
copyNum_LogV = 2000;                     // Set ID number of LogV
new G4PVPlacement{ trans3D_LogV, "PhysVol_WaterCont", logVol_WaterCont,
                  physVol_World, false, copyNum_LogV, true };

// Return the physical volume of 'World'
return physVol_World;
}
```

水・ボリウムの中にアルミ・ボリウムを配置

- logical volumeをlogical volumeに設置する場合、このG4PVPlacementの関数シグネチャを使う

アルミ・ボリウムをWorldボリウムの中に配置

- logical volumeをphysical volumeに設置する場合、このG4PVPlacementの関数シグネチャを使う

P06_Geometry/BGO_One

円筒型BGO検出器ジオメトリ

演習トピックス

[概要] [P06_Geometry](#) プログラムを使い、ジオメトリの記述方法を学ぶ

[目次]

1. [P06_Geometry/WaterBox](#)
 - ジオメトリの記述方法の基礎を学ぶ
2. [P06_Geometry/WaterContainer](#)
 - 容器に入った水の記述
3. [P06_Geometry/BGO_One](#)
 - 円筒型BGO検出器ジオメトリ
4. [P06_Geometry/BGO_Two](#)
 - 円筒型BGO検出器ジオメトリ: Logical Volumeの複数回使用
5. [P06_Geometry/Pixel_One](#)
 - Pixel検出器ジオメトリ: Replicaの使用



P06_Geometry/BGO_Oneプログラムの概要

■ 演習プログラムの目的

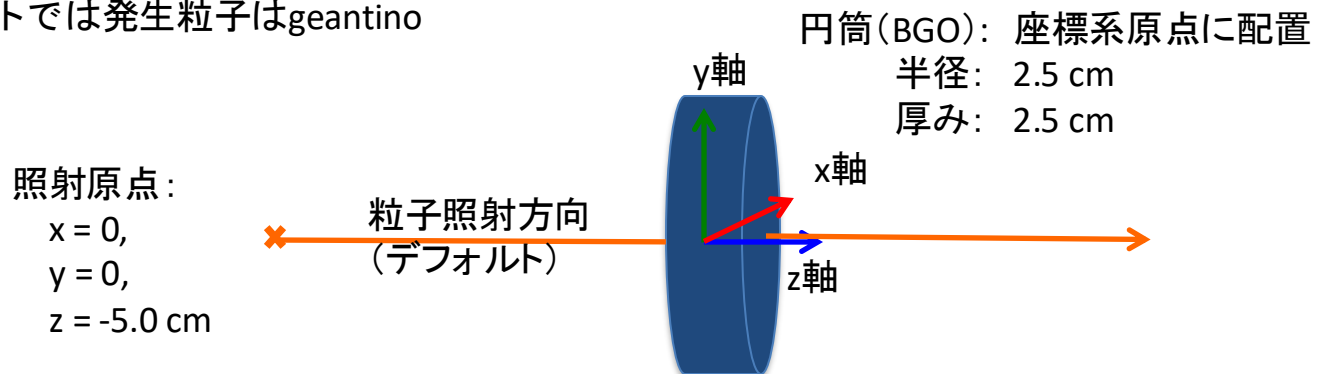
- 「円柱のBGO検出器」を記述したプログラムをビルドして実行する
- 新たなボリューム形状を使ってみる

■ プログラムの構成

- mainプログラム: 前演習と同一
- ジオメトリ: 円柱BGO (下図参照)
- PhysicsList: 前演習と同一
- PrimaryGenerator: 前演習と同一

■ 組み込むジオメトリと粒子発生機能

- 粒子を一つだけBGOに向けて照射
- デフォルトでは発生粒子はgeantino



P06_Geometry/BGO_One プログラムのビルド

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- 前演習のWaterContainerジオメトリがBGO_Oneジオメトリに置き換えられている
- Geometry.cc以外のコードは前演習と同一

課題:2 P06_Geometry/BGO_Oneのソースコードをビルドして実行する

1) P06_Geometry/BGO_Oneのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P06_Geometry/WaterContainer
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P06_Geometry/BGO_Oneプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac  
/run/beamOn 1
```

または、Qtのファイルアイコンをクリック

- 右図の様にgenantinoの飛跡が見えることを確認

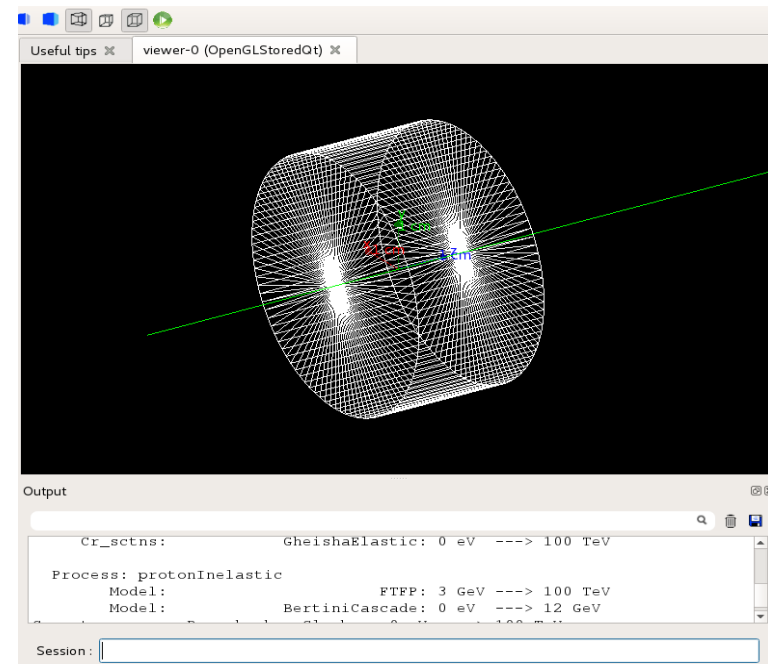
2) Qtコマンド窓で粒子の種類、エネルギーを変えて画面の表示、tracking情報などをみる

```
/gun/particle proton  
/gun/energy 60 MeV  
/run/beamOn 1
```

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
Geometryクラスのコードを読み、実装手法を学ぶ



P06_Geometry/BGO_One: Geometry.ccの内容

課題:1 BGO検出器 のジオメトリ定義のソース ファイルを理解する

```
$ cd ../source/src
$ less Geometry.cc
```

```
//-----
G4VPhysicalVolume* Geometry::Construct()
//-----
{
== world volume作成部分のCodeは不変なので省略 ==

// Define 'BGO Detector'
// Define the shape of solid
G4double radius_BGO = 2.5*cm;
G4double leng_Z_BGO = 2.5*cm;
auto solid_BGO = new G4Tubs{ "Solid_BGO", 0., radius_BGO, leng_Z_BGO/2.0, 0.,
                             360.*deg };

// Define the logical volume
G4Material* materi_BGO = materi_Man->FindOrBuildMaterial( "G4_BGO" );
auto logVol_BGO = new G4LogicalVolume{ solid_BGO, materi_BGO, "LogVol_BGO",
                                       0, 0, 0 };

// Placement of 'BGO Detector' into 'World Volume'
// Create G4Transform3D to define rotation/translation
G4double pos_X_LogV = 0.0*cm;          // X-location LogV
G4double pos_Y_LogV = 0.0*cm;          // Y-location LogV
G4double pos_Z_LogV = 0.0*cm;          // Z-location LogV
auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV };
auto rotMtrx_LogV = G4RotationMatrix{};
auto trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV };

// Install 'BGO Detector'
G4int copyNum_LogV = 1000;              // Set ID number of LogV
new G4PVPlacement{ trans3D_LogV, "PhysVol_BGO", logVol_BGO,
                  physVol_World, false, copyNum_LogV, true };

// Return the physical volume of 'World'
return physVol_World;
}
```

BGO検出器の作成開始

BGO検出器のsolid(tube)を作成

BGO検出器のlogical volume(BGO)を作成

BGO検出器のWorldへの配置

以前の課題(WaterBoxの配置)と
全く同じ手続き

worldのphysical volumeのポインターを返す

P06_Geometry/BGO_Two

円筒型BGO検出器ジオメトリ： Logical Volumeの複数回使用

演習トピックス

[概要] [P06_Geometry](#)プログラムを使い、ジオメトリの記述方法を学ぶ

[目次]

1. [P06_Geometry/WaterBox](#)
 - ジオメトリの記述方法の基礎を学ぶ
2. [P06_Geometry/WaterContainer](#)
 - 容器に入った水の記述
3. [P06_Geometry/BGO_One](#)
 - 円筒型BGO検出器ジオメトリ
4. [P06_Geometry/BGO_Two](#)
 - 円筒型BGO検出器ジオメトリ： Logical Volumeの複数回使用
5. [P06_Geometry/Pixel_One](#)
 - Pixel検出器ジオメトリ： Replicaの使用



P06_Geometry/BGO_Twoプログラムの概要

■ 演習プログラムの目的

[注] コマンド入力は「tabキー」補完機能を使う

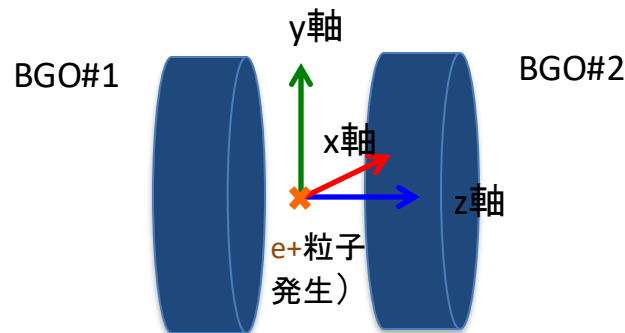
- 前演習のP06_Geometry/BGO_Oneで使ったGeometryファイルをもとに、BGO検出器を1台追加し、プログラムをビルドして実行する
- 新たに作るプログラムはP06_Geometry/BGO_Twoとよぶ
- 演習を実行することで、同じlogical volumeを複数回使いジオメトリを定義する方法を学ぶ

■ プログラムの構成

- mainプログラム: 前演習と同一
- ジオメトリ: 二つの円柱BGO (下図参照)
- PhysicsList: 前演習と同一
- PrimaryGenerator: 前演習と同一

■ 組み込むジオメトリと粒子発生機能

- 静止 e^+ 粒子を二つのBGOの間に発生(UIコマンドを入力する)



円筒(BGO):

半径: 2.5 cm

厚み: 2.5 cm

BGOの中心配置位置:

BGO#1: $X=Y=0\text{cm}$, $Z=-2.0\text{cm}$

BGO#2: $X=Y=0\text{cm}$, $Z=+2.0\text{cm}$

P06_Geometry/BGO_Twoプログラムのビルド

[注] コマンド入力は「tabキー」補完機能を使う

課題:1 プログラム構成を理解する

- 前演習のBGO_Oneジオメトリにもう一つのBGOが追加されたGeometry.ccが組み込まれている
- Geometry.cc以外のコードは前演習と同一

課題:2 P06_Geometry/BGO_Twoのソースコードをビルドして実行する

1) P06_Geometry/BGO_Twoのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P06_Geometry/BGO_Two
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ)に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P06_Geometry/BGO_Twoプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac  
/run/beamOn 1
```

または、Qtのファイルアイコンをクリック

2) 次に、以下のコマンドを入力して、**e+**を原点に発生させる

```
/gun/particle e+  
/gun/energy 0.0 MeV  
/gun/position 0.0 0.0 0.0 cm  
/run/beamOn 1
```

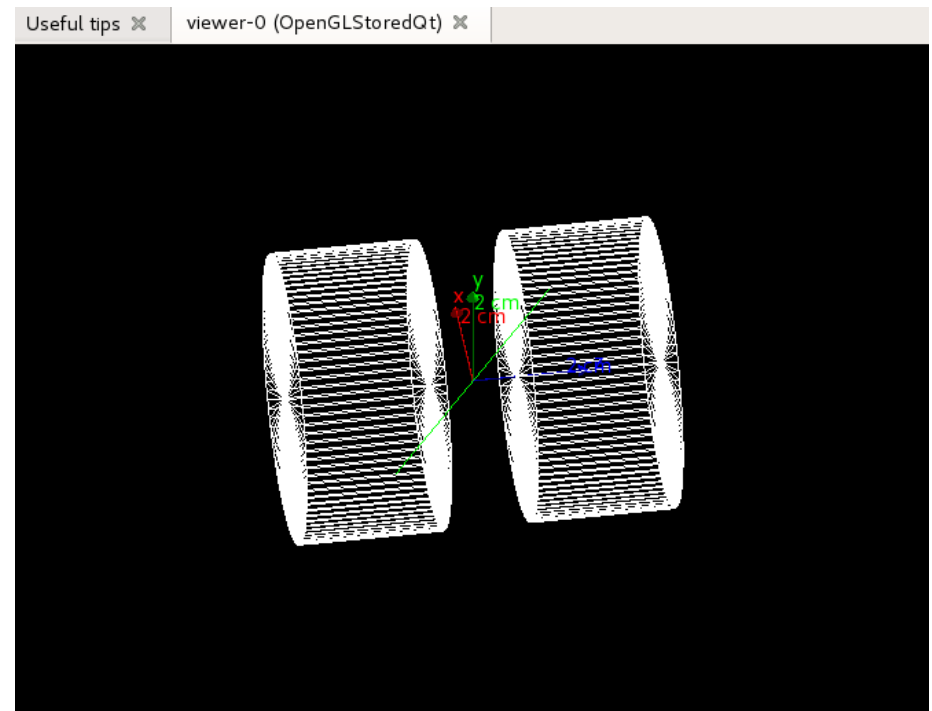
● 右図はその出力:

e+が消滅してback-to-backに2 γ が出ている

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
Geometryクラスのコードを読み、実装手法を学ぶ



P06_Geometry/BGO_Two: Geometry.ccの内容

課題:1 前演習(BGO_One)のGeometry.ccとBGO_TwoのGeometry.ccをcolordiffコマンドで比較

```
$ cd ../source/src
$ colordiff -y --width=160 ../../../BGO_One/source/src/Geometry.cc Geometry.cc
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から新たに追加された箇所を確認する

//----- G4VPhysicalVolume* Geometry::Construct() //-----	変更前ファイル	//----- G4VPhysicalVolume* Geometry::Construct() //-----	変更後ファイル
== world volume作成部分のCodeは不変なので省略 ==			
// Define 'BGO Detector' // Define the shape of solid G4double radius_BGO = 2.5*cm; G4double leng_Z_BGO = 2.5*cm; auto solid_BGO = new G4Tubs{ "Solid_BGO", 0., radius_BGO, leng_Z_BGO, 360.*deg }; // Define logical volume G4Material* materi_BGO = materi_Man->FindOrBuildMaterial("G4_BGO" auto logVol_BGO = new G4LogicalVolume{ solid_BGO, materi_BGO, "LogV" 0, 0, 0 };		// Define 'BGO Detector' // Define the shape of solid G4double radius_BGO = 2.5*cm; G4double leng_Z_BGO = 2.5*cm; auto solid_BGO = new G4Tubs{ "Solid_BGO", 0., radius_BGO, leng_Z_BGO, 360.*deg }; // Define logical volume G4Material* materi_BGO = materi_Man->FindOrBuildMaterial("G4_BGO" auto logVol_BGO = new G4LogicalVolume{ solid_BGO, materi_BGO, "LogV" 0, 0, 0 };	
// Placement of 'BGO Detector' into 'World Volume' // Create G4Transform3D to define rotation/translation G4double pos_X_LogV = 0.0*cm; // X-location LogV G4double pos_Y_LogV = 0.0*cm; // Y-location LogV G4double pos_Z_LogV = 0.0*cm; // Z-location LogV auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV }; auto rotMtrx_LogV = G4RotationMatrix{}; auto trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV }	変更	// Placement of two 'BGO Detectors' into 'World Volume' // Create G4Transform3D to define rotation/translation G4double pos_X_LogV = 0.0*cm; // X-location LogV G4double pos_Y_LogV = 0.0*cm; // Y-location LogV G4double pos_Z_LogV = -2.0*cm; // Z-location LogV auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV }; auto rotMtrx_LogV = G4RotationMatrix{}; auto trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV }	変更
// Install 'BGO Detector' G4int copyNum_LogV = 1000; // Set ID number of LogV	変更	// Install 1st 'BGO Detector' G4int copyNum_LogV = 1000; // Set ID number of LogV new G4PVPlacement{ trans3D_LogV, "PhysVol_BGO", logVol_BGO, physVol_World, false, copyNum_LogV, true };	追加
new G4PVPlacement{ trans3D_LogV, "PhysVol_BGO", logVol_BGO, physVol_World, false, copyNum_LogV, true };		// Install 2nd 'BGO Detector' copyNum_LogV = 2000; // Set ID number of LogV pos_Z_LogV = +2.0*cm; // Z-location LogV threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV }; trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV };	
// Return the physical volume of 'World' return physVol_World; }		// Return the physical volume of 'World' return physVol_World; }	

P06_Geometry/BGO_Twoプログラムの留意点

課題:1 BGO_Twoプログラムのジオメトリ記述で使われている手法を理解する

- 1) 同一のvolumeを幾度も配置する場合、volumeを繰り返し定義せずに、そのlogical volumeを使いまわす
→複雑なジオメトリをメモリ消費を少なく記述するキーポイント、
- 2) BGO_Twoでは、BGOのlogical volumeを一つ定義し、それを使いまわして、2台のBGO検出器を配置

BGO_TwoプログラムのGeometry.ccの最終部分

```
// Define 'BGO Detector'
// Define the shape of solid
G4double radius_BGO = 2.5*cm;
G4double leng_Z_BGO = 2.5*cm;
auto solid_BGO = new G4Tubs{ "Solid_BGO", 0., radius_BGO, leng_Z_BGO/2.0, 0.,
                             360.*deg };

// Define the logical volume
G4Material* materi_BGO = materi_Man->FindOrBuildMaterial( "G4_BGO" );
auto logVol_BGO = new G4LogicalVolume{ solid_BGO, materi_BGO, "LogVol_BGO",
                                       0, 0, 0 };

// Placement of two 'BGO Detectors' into 'World Volume'
// Create G4Transform3D to define rotation/translation
G4double pos_X_LogV = 0.0*cm;           // X-location LogV
G4double pos_Y_LogV = 0.0*cm;           // Y-location LogV
G4double pos_Z_LogV = -2.0*cm;          // Z-location LogV
auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV };
auto rotMtrx_LogV = G4RotationMatrix{};
auto trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV };

// Install the 1st 'BGO Detector'
G4int copyNum_LogV = 1000;               // Set ID number of LogV
new G4PVPlacement{ trans3D_LogV, "PhysVol_BGO", logVol_BGO,
                  physVol_World, false, copyNum_LogV, true };

// Install the 2nd 'BGO Detector'
copyNum_LogV = 2000;                     // Set ID number of LogV
pos_Z_LogV = +2.0*cm;                     // Z-location LogV
threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV };
trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV };

new G4PVPlacement{ trans3D_LogV, "PhysVol_BGO", logVol_BGO,
                  physVol_World, false, copyNum_LogV, true };

// Return the physical volume of 'World'
return physVol_World;
}
```

← BGO検出器のlogical volume
を定義

← 1台目のBGO検出器を上で定義した
logical volumeを使い配置

← 2台目のBGO検出器を上で定義した
logical volumeを使い配置

P06_Geometry/Pixel_One

Pixel検出器ジオメトリ： Replicaの使用

演習トピックス

[概要] [P06_Geometry](#)プログラムを使い、ジオメトリの記述方法を学ぶ

[目次]

1. [P06_Geometry/WaterBox](#)
 - ジオメトリの記述方法の基礎を学ぶ
2. [P06_Geometry/WaterContainer](#)
 - 容器に入った水の記述
3. [P06_Geometry/BGO_One](#)
 - 円筒型BGO検出器ジオメトリ
4. [P06_Geometry/BGO_Two](#)
 - 円筒型BGO検出器ジオメトリ： Logical Volumeの複数回使用
5. [P06_Geometry/Pixel_One](#)
 - Pixel検出器ジオメトリ： Replicaの使用



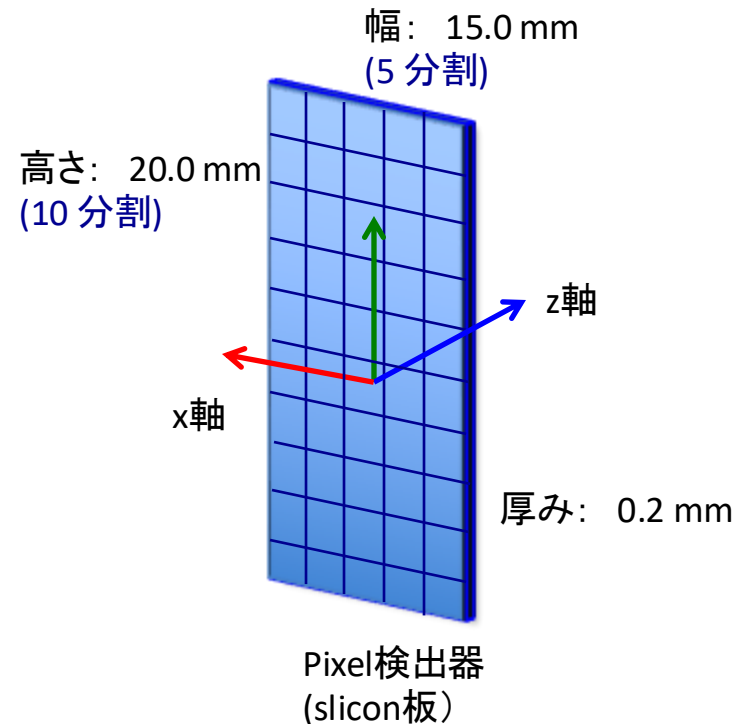
P06_Geometry/Pixel_Oneプログラムの概要

■ 演習プログラムの目的

- Pixel検出器のジオメトリをレプリカ配置で記述したプログラムをビルドして実行する
- レプリカを用いると、検出器の個々の構成要素を個別に配置せずに、自動的にまとめて配置できることを学ぶ

■ プログラムの構成

- mainプログラム: 前演習と同一
- ジオメトリ: 一枚のPixel検出器
- PhysicsList: 前演習と同一
- PrimaryGenerator: 前演習と同一



P06_Geometry/Pixel_One プログラムのビルド

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- 前演習のジオメトリをPixel_Oneジオメトリに置き換える
- Geometry.cc以外のコードは前演習と同一

課題:2 P06_Geometry/Pixel_Oneのソースコードをビルドして実行する

1) P06_Geometry/Pixel_Oneのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P06_Geometry/Pixel_One
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P06_Geometry/Pixel_Oneプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

または、Qtのファイルアイコンをクリック

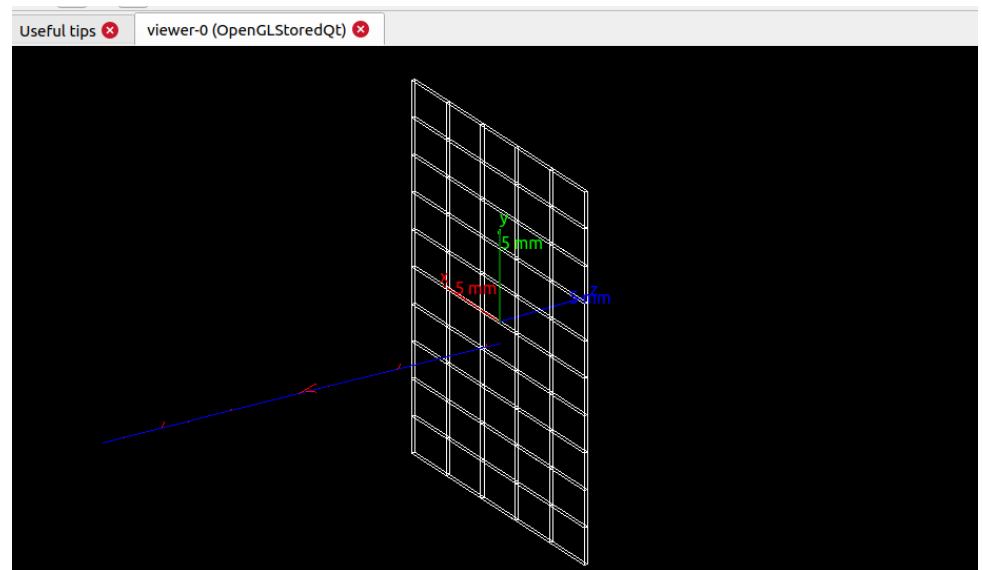
2) Qtコマンド窓に以下のコマンドを入力

```
/gun/particle proton  
/gun/energy 5 MeV  
/run/beamOn 1
```

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
Geometryクラスのコードを読み、実装手
法を学ぶ

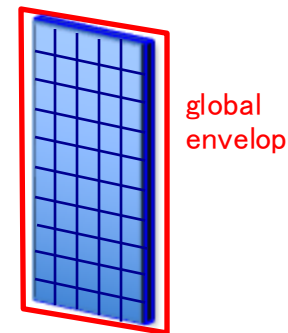


P06_Geometry/Pixel_One: Geometry.ccの内容

課題:2 P06_Geometry: Pixel_Oneの内容を理解する

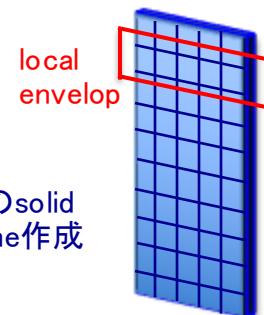
```
$ cd ../source/src  
$ less Geometry.cc
```

```
//-----  
G4VPhysicalVolume* Geometry::Construct()  
//-----  
{  
    == world volume作成部分のCodeは不変なので省略 ==  
  
    // Define 'Pixel Detector' - Global Envelop Pixel検出器の外形枠(global envelop)作成  
    // Define the shape of the global envelop  
    G4double leng_X_PixEnvG = 15.0*mm;    // X-full-length of pixel: global envelop  
    G4double leng_Y_PixEnvG = 20.0*mm;    // Y-full-length of pixel: global envelop  
    G4double leng_Z_PixEnvG = 0.2*mm;    // Z-full-length of pixel: global envelop  
    auto solid_PixEnvG = new G4Box{ "Solid_PixEnvG", leng_X_PixEnvG/2.0,  
                                    leng_Y_PixEnvG/2.0, leng_Z_PixEnvG/2.0 };  
  
    // Define the logical volume of the global envelop  
    G4Material* materi_PixEnvG = materi_Man->FindOrBuildMaterial( "G4_AIR" );  
    auto logVol_PixEnvG = new G4LogicalVolume{ solid_PixEnvG, materi_PixEnvG,  
                                                "LogVol_PixEnvG" };  
    logVol_PixEnvG->SetVisAttributes ( G4VisAttributes::Invisible );  
  
    // Define 'Pixel Detector' - Local Envelop (divided the global envelop in Y-direction) Pixel検出器の外形枠のY軸方向を10分割  
    // Define the shape of the local envelop  
    G4int nDiv_Y = 10;    // Number of divisions in Y-direction  
    G4double leng_X_PixEnvL = leng_X_PixEnvG;    // X-full-length of pixel: local envelop  
    G4double leng_Y_PixEnvL = leng_Y_PixEnvG/nDiv_Y;    // Y-full-length of pixel: local envelop  
    G4double leng_Z_PixEnvL = leng_Z_PixEnvG;    // Z-full-length of pixel: local envelop  
    auto solid_PixEnvL = new G4Box{ "Solid_PixEnvL", leng_X_PixEnvL/2.0,  
                                    leng_Y_PixEnvL/2.0, leng_Z_PixEnvL/2.0 };  
  
    // Define the logical volume of the local envelop  
    G4Material* materi_PixEnvL = materi_Man->FindOrBuildMaterial( "G4_AIR" );  
    auto logVol_PixEnvL = new G4LogicalVolume{ solid_PixEnvL, materi_PixEnvL,  
                                                "LogVol_PixEnvL" };  
    logVol_PixEnvL->SetVisAttributes ( G4VisAttributes::Invisible );  
  
    // Placement of the local envelop to the global envelop using Replica  
    new G4PVReplica{ "PhysVol_PixEnvL", logVol_PixEnvL, logVol_PixEnvG, kYAxis,  
                    nDiv_Y, leng_Y_PixEnvL };  
}
```



global envelop

global envelopのsolidとlogical volume作成



local envelop

local envelopのsolidとlogical volume作成

上で定義されたlocal envelop (logical volume) 10個をレプリカを使い、global envelopに配置
配置はG4PVReplicaを使用

P06_Geometry/Pixel_One: Geomety.ccの内容 (続き)

課題:4 Geometry.cc_Pixelの内容を理解する

前のコードの続き

```
// Define 'Pixel Detector' - Pixel Element (divided the local envelop in X-direction)
// Define the shape of the pixel element
G4int nDiv_X = 5; // Number of divisions in Y-direction
G4double leng_X_Pixelmt = leng_X_PixelEnvG/nDiv_X; // X-full-length of pixel: pixel element
G4double leng_Y_Pixelmt = leng_Y_PixelEnvG/nDiv_Y; // Y-full-length of pixel: pixel element
G4double leng_Z_Pixelmt = leng_Z_PixelEnvG; // Z-full-length of pixel: pixel element
auto solid_Pixelmt = new G4Box{ "Solid_Pixelmt", leng_X_Pixelmt/2.0,
                                leng_Y_Pixelmt/2.0, leng_Z_Pixelmt/2.0 };

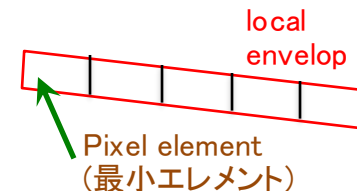
// Define logical volume of the pixel element
G4Material* materi_Pixelmt = materi_Man->FindOrBuildMaterial( "G4_SILICON_DIOXIDE" );
auto logVol_Pixelmt = new G4LogicalVolume{ solid_Pixelmt, materi_Pixelmt,
                                           "LogVol_Pixelmt" };
//logVol_Pixelmt->SetVisAttributes ( G4VisAttributes::Invisible );

// Placement of pixel elements to the local envelop using Replica
new G4PVReplica{ "PhysVol_Pixelmt", logVol_Pixelmt, logVol_PixelEnvL, kXAxis,
                nDiv_X, leng_X_Pixelmt };

// Placement of the 'Pixel Detector' to the world: Put the 'global envelop'
G4double pos_X_LogV_PixelEnvG = 0.0*cm; // X-location LogV_PixelEnvG
G4double pos_Y_LogV_PixelEnvG = 0.0*cm; // Y-location LogV_PixelEnvG
G4double pos_Z_LogV_PixelEnvG = 0.0*cm; // Z-location LogV_PixelEnvG
auto threeVect_LogV_PixelEnvG = G4ThreeVector{ pos_X_LogV_PixelEnvG,
                                                pos_Y_LogV_PixelEnvG, pos_Z_LogV_PixelEnvG };
auto rotMtrix_LogV_PixelEnvG = G4RotationMatrix{};
auto trans3D_LogV_PixelEnvG = G4Transform3D{ rotMtrix_LogV_PixelEnvG, threeVect_LogV_PixelEnvG };

G4int copyNum_LogV_PixelEnvG = 1000; // Set ID number of LogV_PixelEnvG
new G4PVPlacement{ trans3D_LogV_PixelEnvG, "PhysVol_PixelEnvG", logVol_PixelEnvG, physVol_World,
                  false, copyNum_LogV_PixelEnvG };

// Return the physical world
return physVol_World;
}
```



Pixelの最小エレメントのsolidとlogical volume作成

Pixelの最小エレメント(logical volume) 5個をレプリカを使い、local envelopに配置
配置はG4PVReplicaを使用

最終的にPixel全体をワールドボリュウムに配置

→ Pixelのglobal volumeを配置すれば良い
配置はG4PVPlacementを使用

演習 終了

自習課題

P06_Geometry/WaterExtruded

ボリウムが部分重複したらどうなるか

P06_Geometry/WaterExtrudedプログラムの概要

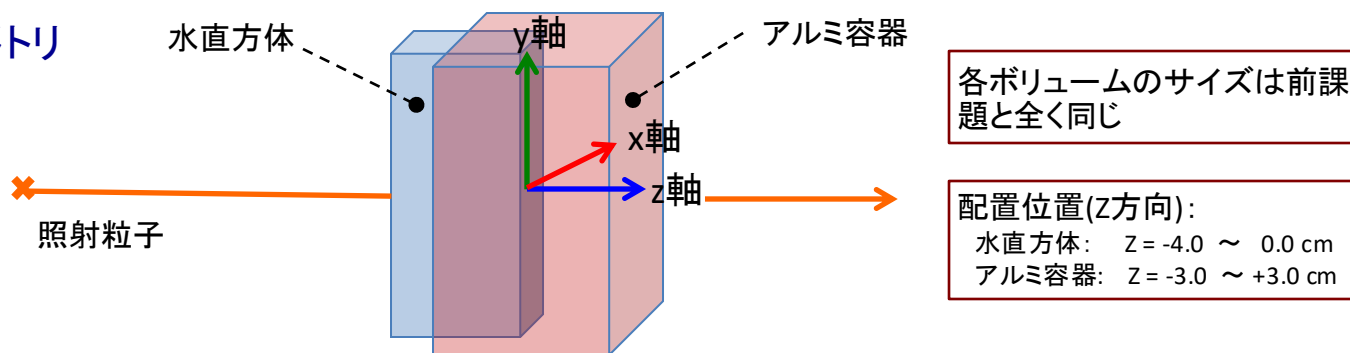
■ 演習プログラムの目的

- ジオメトリ記述ではvolumeが他のvolumeと部分的に重なることは禁止されている
 - ➔ 記述を間違っって重なりを作ってしまうとどうなるかを学ぶ
 - ➔ ジオメトリ記述でvolumeの重なりを作らない重要性を学ぶ
- 前課題のアルミコンテナと水boxのジオメトリ記述を変更し、部分的な重なりを作りgeantinoを照射すると何が生じるかを知る
- 新たに作るプログラムはP06_Geometry: WaterExtrudedとよぶ

■ プログラムの構成

- mainプログラム: WaterContainerと同一
- ジオメトリ: 水がアルミ直方体容器から突き出ている (下図参照)
- PhysicsList: WaterContainerと同一
- PrimaryGenerator: WaterContainerと同一

■ 組み込むジオメトリ



P06_Geometry/WaterExtrudedプログラムの作成

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- P06_Geometry/WaterContainerコードのGeometry.ccでwater boxをcontainerからはみ出させる
- Geometry.cc以外のコードは前演習と同一

課題:2 P06_Geometry/WaterExtrudedのソースコードをビルドして実行する

1) P06_Geometry/WaterExtrudedのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P06_Geometry/WaterExtruded
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

4) Qt窓が表示されるが、端末にはgeometry設定の警告メッセージが出力されていることを確認する

← 警告内容の詳細は3ページ後のスライド参照

P06_Geometry/WaterExtrudedプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac  
/run/beamOn 1
```

または、Qtのファイルアイコンをクリック

- Geometry設定の警告があるにも関わらず、右図の様にgeantinoの飛跡が見えることを確認

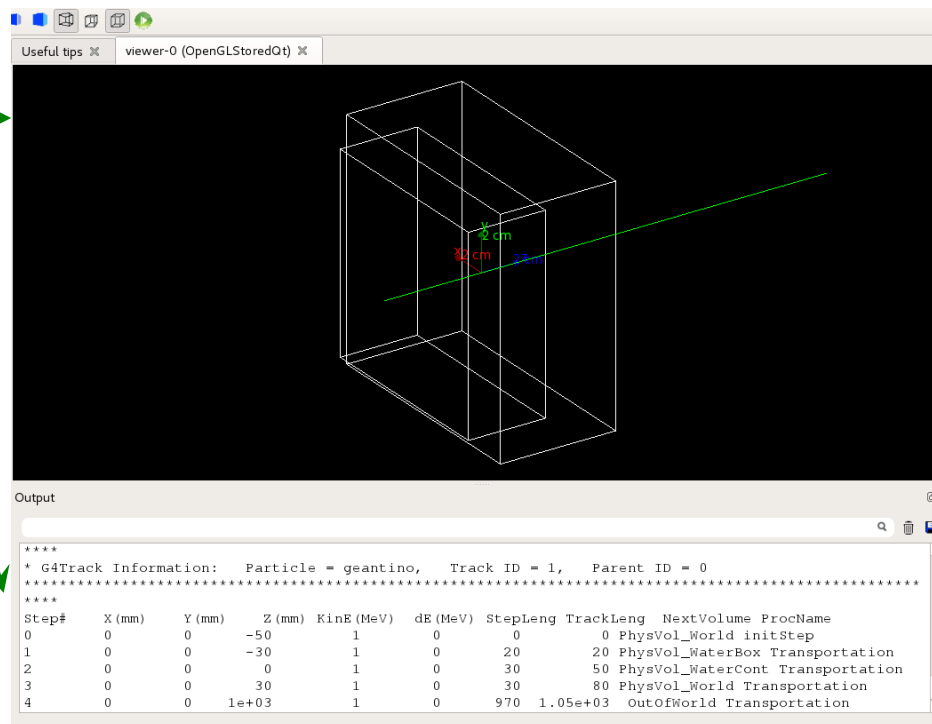
- しかし、geantinoのtracking情報には異常がある

← 異常の詳細は次々ページのスライド参照

2) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
Geometryクラスのコードを読み、実装手法を学ぶ



P06_Geometry/WaterExtruded: Geometry.ccの内容

課題:1 WaterContainerのGeometry.ccとWaterExtrudedのGeometry.ccをcolordiffコマンドで比較

```
$ cd ../source/src
$ colordiff -y --width=160 ../../WaterContainer/source/src/Geometry.cc Geometry.cc
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から新たに追加された箇所を確認する

変更前ファイル	変更後ファイル
== 以下はGeometry.ccコードの最終部分 ==	
<pre>// Placement of 'Water Box' into 'Water Container' // Create G4Transform3D to define rotation/translation G4double pos_X_LogV = 0.0*cm; // X-location LogV G4double pos_Y_LogV = 0.0*cm; // Y-location LogV G4double pos_Z_LogV = 0.0*cm; // Z-location LogV auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV }; auto rotMtrx_LogV = G4RotationMatrix{}; auto trans3D_LogV = G4Transform3D{ rotMtrx_LogV, threeVect_LogV }; // Install 'Water Box'</pre>	<pre>// Placement of 'Water Box' into 'Water Container' // Create G4Transform3D to define rotation/translation G4double pos_X_LogV = 0.0*cm; // X-location LogV G4double pos_Y_LogV = 0.0*cm; // Y-location LogV G4double pos_Z_LogV = 0.0*cm; // Z-location LogV auto threeVect_LogV = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV }; auto rotMtrx_LogV = G4RotationMatrix{}; auto trans3D_LogV = G4Transform3D(rotMtrx_LogV, threeVect_LogV); // Install 'Water Box' G4double delZ = -2.0*cm; auto threeVect_LogV_tmp = G4ThreeVector{ pos_X_LogV, pos_Y_LogV, pos_Z_LogV }; auto trans3D_LogV_tmp = G4Transform3D(rotMtrx_LogV, threeVect_LogV_tmp);</pre>
<pre>G4int copyNum_LogV = 1000; // Set ID number of LogV new G4PVPlacement{ trans3D_LogV, logVol_WaterBox, "PhysVol_WaterBox", 変更 logVol_WaterCont, false, copyNum_LogV, true };</pre>	<pre>G4int copyNum_LogV = 1000; // Set ID number of LogV new G4PVPlacement{ trans3D_LogV_tmp, logVol_WaterBox, "PhysVol_WaterBox", logVol_WaterCont, false, copyNum_LogV, true };</pre>
<pre>// Placement of 'Water Container' into 'World Volume' // Install 'Water Container' - reuse G4Transform3D for 'Water Box' copyNum_LogV = 2000; // Set ID number of LogV new G4PVPlacement{ trans3D_LogV, "PhysVol_WaterCont", logVol_WaterCont, physVol_World, false, copyNum_LogV, true };</pre>	<pre>// Placement of 'Water Container' into 'World Volume' // Install 'Water Container' - reuse G4Transform3D for 'Water Box' copyNum_LogV = 2000; // Set ID number of LogV new G4PVPlacement{ trans3D_LogV, "PhysVol_WaterCont", logVol_WaterCont, physVol_World, false, copyNum_LogV, true };</pre>
<pre>// Return the physical volume of 'World' return physVol_World; }</pre>	<pre>// Return the physical volume of 'World' return physVol_World; }</pre>

[注] 変更点のポイントは以下のとおり:

- Z軸方向に-2.0cm平行移動のG4Transform3Dオブジェクト(trans3D_LogV_tmp)を作る
- 水boxをアルミ容器に設置する際、このG4Transform3Dオブジェクトに置き換える

P06_Geometry/WaterExtrudedプログラムの留意点

課題: 1 WaterExtrudedプログラムの実行で生じている問題を理解する

1) ボリュームが部分的に重なって置かれていることの警告メッセージを知る

- WaterExtrudeプログラムを実行した時点で、**ターミナル窓**に以下の様な**警告メッセージ**が表示される

```
*****
Geant4 version Name: geant4-11-03 [MT] (6-December-2024)
Copyright : Geant4 Collaboration
References : NIM A 506 (2003), 250-303
            : IEEE-TNS 53 (2006), 270-278
            : NIM A 835 (2016), 186-225
WWW : http://geant4.org/
*****
```

```
<<< Geant4 Physics List simulation engine: FTFP_BERT 2.0
```

```
Checking overlaps for volume PhysVol_WaterBox ...
```

```
----- WWWWW ----- G4Exception-START ----- WWWWW -----
```

```
*** G4Exception : GeomVol1002
```

```
issued by : G4PVPlacement::CheckOverlaps()
```

```
Overlap with mother volume !
```

```
Overlap is detected for volume PhysVol_WaterBox
```

```
with its mother volume LogVol_WaterCont
```

```
at mother local point (49.5947,11.7751,-40), overlapping by at least: 1 cm
```

```
NOTE: Reached maximum fixed number -1- of overlaps reports for this volume !
```

```
*** This is just a warning message. ***
```

```
----- WWWWW ----- G4Exception-END ----- WWWWW -----
```

- G4PVPlacementはボリームの部分重なるのチェック機能を持っている
- チェック機能を有効にするにはG4PVPlacementの最後の引数を 'true' をセットする

```
PVPlacement( ..... , true);
```

[注] チェック機能は常にセットすることを推奨

← チェック機能からの警告メッセージ

2) geantinoのtracking情報

```
*****
* G4Track Information: Particle = geantino, Track ID = 1, Parent ID = 0
*****
Step#   X(mm)   Y(mm)   Z(mm)   KinE(MeV)   dE(MeV)   StepLeng   TrackLeng   NextVolume   ProcName
0       0       0       -50     1           0          0          0          PhysVol_World initStep
1       0       0       -30     1           0          20         20         PhysVol_WaterBox Transportation
2       0       0       0       1           0          30         50         PhysVol_WaterCont Transportation
3       0       0       30     1           0          30         80         PhysVol_World Transportation
4       0       0       1e+03  1           0          970        1.05e+03  OutOfWorld Transportation
```

- 水直方体Z面はZ=-40mmから始まるが、geantinoはこの面を認識しない
- アルミZ面のZ=-30mmは認識されるが、その面は水直方体に含まれているので媒質は水となる

- 水直方体のZ=-40 ~ -30mmがtrackingで無視されるのは、その部分がアルミ容器から突き出ているため
- 原因: 親volumeは直下の子volumeは認識するが、孫以下のvolumeは認識しない ➡ ボリームの部分重複配置は禁止

P06_Geometry/Pixel_Four

4台のPixel検出器の配置

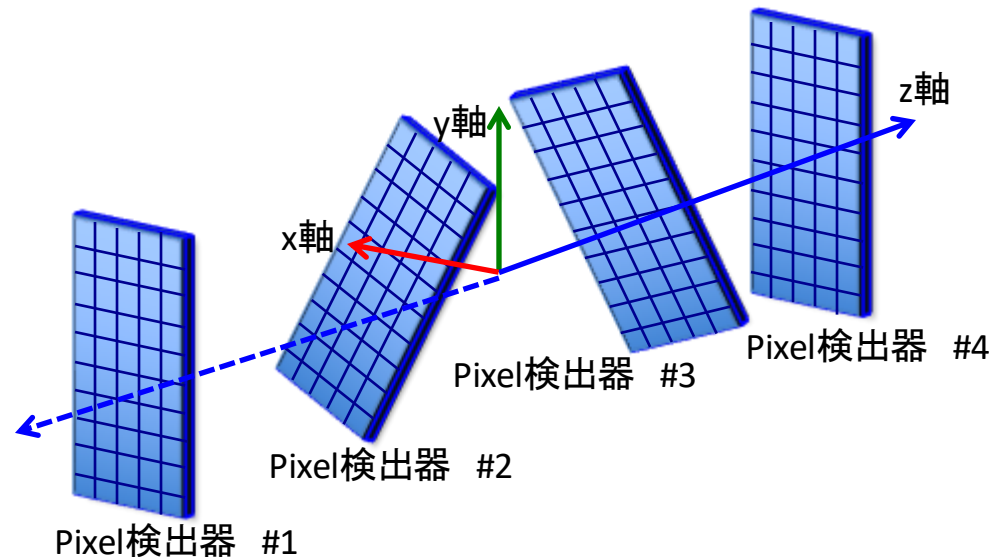
P06_Geometry/Pixel_Fourプログラムの概要

■ 演習プログラムの目的

- 演習のP06_Geometry/Pixel_Oneで学んだGeometryファイルをもとに、Pixel検出器を3台追加し、プログラムをビルドして実行する
- Pixel検出器を4台を下図のように配置し、pixel検出器を使った場合に生じる粒子飛跡再構成の不確定性を解決できるようにする
- 検出器を移動・回転させて配置させるために、G4Transform3Dをどう使うかを学ぶ
- 新たに作るプログラムはP06_Geometry: Pixel_Fourとよぶ

■ プログラムの構成

- mainプログラム: Pixel_Oneと同一
- ジオメトリ: 4枚のPixel検出器
- PhysicsList: Pixel_Oneと同一
- PrimaryGenerator: Pixel_Oneと同一



P06_Geometry/Pixel_Four プログラムのビルド

課題:1 プログラム構成を理解する

[注] コマンド入力は「tabキー」補完機能を使う

- P06_Geometry/Pixel_OneコードのGeometry.ccに3台のPixelを追加する
- Geometry.cc以外のコードは前演習と同一

課題:2 P06_Geometry/Pixel_Fourのソースコードをビルドして実行する

1) P06_Geometry/Pixel_Oneのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P06_Geometry/Pixel_Four
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P06_Geometry: Pixel_Fourプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac  
/run/beamOn 1
```

または、Qtのファイルアイコンをクリック

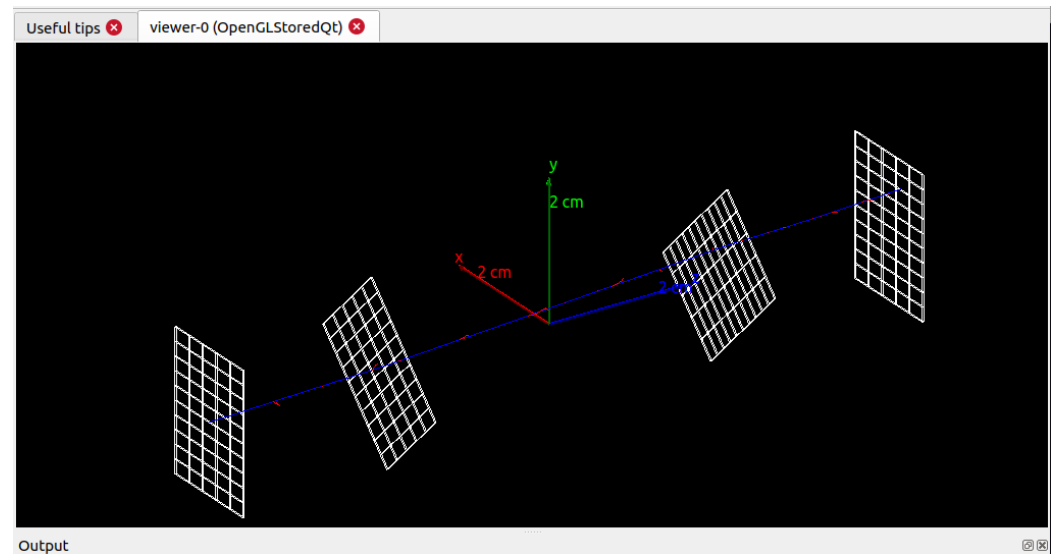
2) Qtコマンド窓に以下のコマンドを入力

```
/gun/particle proton  
/gun/energy 10 MeV  
/run/beamOn 1
```

2) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
[Geometryクラス](#)のコードを読み、実装手
法を学ぶ



P06_Geometry/Pixel_Four: Geometry.ccの内容

課題:1 Pixel_OneのGeometry.ccとPixel_FourのGeometry.ccをcolordiffコマンドで比較

```
$ cd ../source/src
$ colordiff -y --width=160 ../../Pixel_One/source/src/Geometry.cc Geometry.cc
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から新たに追加された箇所を確認する

変更前ファイル	変更後ファイル
<pre>//----- G4VPhysicalVolume* Geometry::Construct() //----- == Pixel検出器のlogical volumeを作るまではコードは変更する必要がない == // Placement of the 'Pixel Detector' to the world: Put the 'global envelop' G4double pos_X_LogV_PixEnvG = 0.0*cm; // X-location LogV_PixEnvG G4double pos_Y_LogV_PixEnvG = 0.0*cm; // Y-location LogV_PixEnvG G4double pos_Z_LogV_PixEnvG = 0.0*cm; // Z-location LogV_PixEnvG auto threeVect_LogV_PixEnvG = G4ThreeVector{ pos_X_LogV_PixEnvG, pos_Y_LogV_PixEnvG, pos_Z_LogV_PixEnvG }; -- この囲まれた部分のコードが右のように変更される -- auto rotMtrx_LogV_PixEnvG = G4RotationMatrix{}; auto trans3D_LogV_PixEnvG = G4Transform3D{ rotMtrx_LogV_PixEnvG, threeVect_LogV_PixEnvG }; G4int copyNum_LogV_PixEnvG = 1000; // Set ID number of LogV_PixEnvG new G4PVPlacement{ trans3D_LogV_PixEnvG, "PhysVol_PixEnvG", logVol_PixEnvG, false, copyNum_LogV_PixEnvG }; // Return the physical volume of 'World' return physVol_World; }</pre>	<pre>//----- G4VPhysicalVolume* Geometry::Construct() //----- // Placement of the 'Pixel Detectors' to the world: Put the 'global envelop' // Z positions and setting angles of four detectors G4double z_pos[4] = { -5.0*cm, -2.5*cm, +2.5*cm, +5.0*cm }; G4double z_ang[4] = { 0.0*deg, -45.0*deg, +45.0*deg, 0.0*deg }; // Placement of the detectors for (G4int id=0; id<4; id++){ G4double pos_X_LogV_PixEnvG = 0.0*cm; // X-location LogV_PixEnvG G4double pos_Y_LogV_PixEnvG = 0.0*cm; // Y-location LogV_PixEnvG G4double pos_Z_LogV_PixEnvG = z_pos[id]; // Z-location LogV_PixEnvG auto threeVect_LogV_PixEnvG = G4ThreeVector{ pos_X_LogV_PixEnvG, pos_Y_LogV_PixEnvG, pos_Z_LogV_PixEnvG }; auto rotMtrx_LogV_PixEnvG = G4RotationMatrix{}; rotMtrx_LogV_PixEnvG.rotateZ{ z_ang[id] }; auto trans3D_LogV_PixEnvG = G4Transform3D{ rotMtrx_LogV_PixEnvG, threeVect_LogV_PixEnvG }; G4int copyNum_LogV_PixEnvG = 1000 + id; // Set ID number new G4PVPlacement{ trans3D_LogV_PixEnvG, "PhysVol_PixEnvG", logVol_PixEnvG, false, copyNum_LogV_PixEnvG, true }; } // Return the physical volume of 'World' return physVol_World; }</pre>